Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Bearbeitungszeitraum: von 20. Januar 2026
bis 20. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: June 15, 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Aufgabensteller, Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in (Firma/Behörde/Hochschule):	OTH Amberg-Weiden
Betreuer in Firma/Behörde:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum: 20. Januar 2026	Abgabedatum: 20. Juni 2026

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

---

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Abstract:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Keywords: LaTeX, Textsatz, Formeln, Graphiken

Contents

1 Foundations **1**
1.1 Neural Networks 1
1.1.1 Neuron 1
1.1.2 Neural Networks 3
1.1.3 Training Neural Networks 6
1.1.4 Evaluation 10

Literaturverzeichnis **12**

Abbildungsverzeichnis **13**

Tabellenverzeichnis **14**

Acronyms

CNN Convolutional Neural Network

MLP Multi-Layer Perceptron

MSE Mean Squared Error

RNN Recurrent Neural Network

Symbole, Formelzeichen und Einheiten

Chapter 1

Foundations

1.1 Neural Networks

Artificial Neural Networks are computational models inspired by the structure and functionality of biological brain. They are designed to learn complex relationships from data and have become a fundamental component of modern machine learning. Neural Networks are capable of approximating highly nonlinear functions and are therefore widely used in applications such as image recognition, natural language processing, forecasting, and control systems. There exist various types of neural network architectures, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), or transformer models, each tailored to specific types of data and tasks. The following sections provide an overview of the basic building blocks of neural networks, their mathematical formulation, and the training process.

1.1.1 Neuron

The basic element of an artificial neural network is the artificial neuron. The concept is motivated by biological neurons in the human brain, which communicate through electrochemical signals. In a simplified model, a biological neuron receives signals from other neurons through dendrites, processes the received information, and transmits an output signal through its axon [1]. Artificial neurons mimic this behavior mathematically by combining several input values into a single output.

History

The first mathematical model of an artificial neuron was proposed by McCulloch and Pitts in 1943 [2]. Their model consisted of a binary threshold unit that produced an output only when the weighted sum of its inputs exceeded a predefined threshold. Although this model did not include a learning mechanism, it established the foundation for later neural network research.

A major extension of this idea was introduced by Rosenblatt in 1958 with the perceptron [3]. In contrast to the McCulloch–Pitts neuron, the perceptron introduced trainable

weights and a learning rule, making it possible to adapt the model parameters directly from data. However, the perceptron still relied on a simple threshold-based activation, which produces a linear decision boundary and was therefore limited to linearly separable classification problems [4].

This limitation was highlighted by Minsky and Papert, who showed that a single perceptron cannot represent functions such as XOR [4]. The development of multilayer neural networks together with differentiable activation functions later overcame this restriction. These advances made it possible to train networks with multiple layers and enabled the modern neural network architectures that are used in deep learning today.

Mathematical Formulation

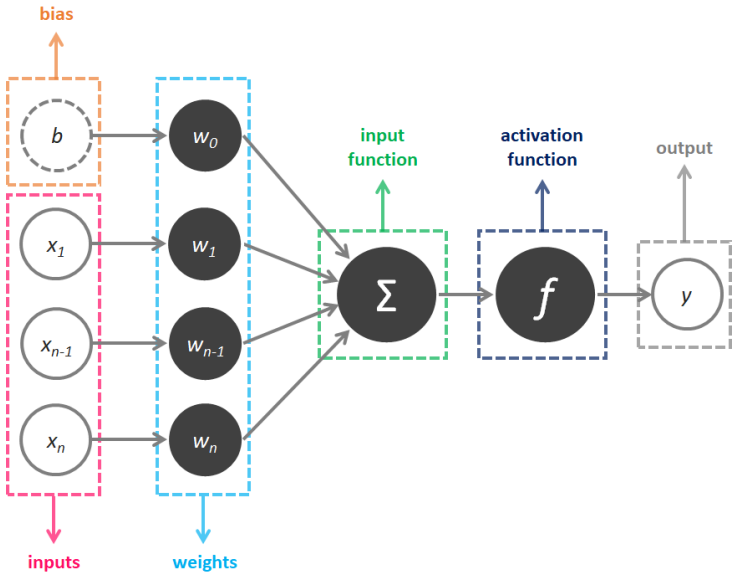

Figure 1.1: Artificial Neuron. Image source: [**xxx**].

A neuron receives an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$, where each vector element x_i is associated with a trainable weight w_i from the corresponding weight vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$. The neuron first computes a weighted sum of its inputs and adds a bias term b, see Figure 1.1. The resulting pre-activation value is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{1.1}$$

The value z represents a linear combination of the input features. To enable the modeling of non-linear relationships, an activation function $\phi(\cdot)$ is subsequently applied, yielding the neuron's output:

$$y = \phi(z) = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{1.2}$$

Without the activation function, a composition of multiple neurons would still correspond to a single linear transformation and therefore could not represent complex non-linear patterns. Common activation functions used in modern neural networks include for example the GeLU, tanh (hyperbolic tangent), and ReLU.

The weighted sum in Equation 1.1 can be expressed more compactly using vector notation and ending in the form of:

$$y = \phi\Big(\mathbf{w}^T\mathbf{x} + b\Big) = \phi\left(\begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b\right) \tag{1.3}$$

This vector formulation serves as the basis for the matrix representation of an entire neural network layer, where multiple neurons are evaluated simultaneously.

1.1.2 Neural Networks

A neural network is a collection of interconnected neurons organized into layers. The input layer receives the feature vector, while the output layer produces the final prediction. The hidden layers between the input and output layers perform intermediate computations that allow the network to learn increasingly abstract representations of the input data. The parameters of the network, including the weights and biases, are learned from data using optimization algorithms such as gradient descent. The ability of neural networks to learn complex functions from data has made them a powerful tool for a wide range of applications in machine learning and artificial intelligence.

To understand the terminology: Artificial neural network is the broadest term, encompassing all brain-inspired, connectionist models. A feedforward neural network is a subcategory where information flows strictly in one direction (from input to output) without forming cycles, but its layers need not be fully connected. The multilayer perceptron is a feedforward network with one or more fully connected (also called dense) hidden layers, where every neuron in one layer connects to every neuron in the next.

Multilayer Perceptron Network

The most common form of such architectures is the *Multi-Layer Perceptron (MLP)*, also referred to as a feedforward neural network, where information flows in one direction from the input layer through one or more fully connected hidden layers to the output layer. In a *Multilayer Perceptron Network*, each neuron in a layer receives the outputs of the neurons in the preceding layer as inputs. In a fully connected layer, every neuron is connected to every neuron in the subsequent layer. The output of a layer therefore serves as the input to the next layer.

The computation of an entire layer can be expressed compactly using matrix notation. Let $\mathbf{x}^{(l-1)}$ denote the output vector of the previous layer, $\mathbf{W}^{(l)}$ the weight matrix, and

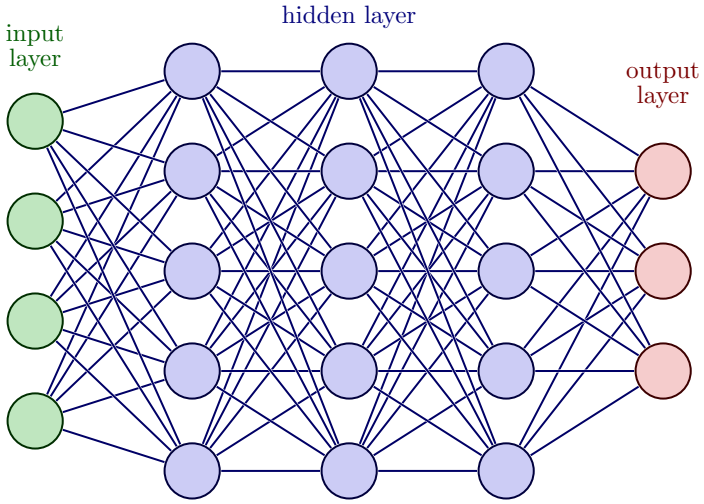

Figure 1.2: Neural Network Architecture. Image source: [**xxx**].

$\mathbf{b}^{(l)}$ the bias vector of layer l. M are the number of neurons in layer l and N the number of neurons in the previous layer $l-1$. The pre-activation vector is given by:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}, \tag{1.4}$$

which can be written explicitly as:

$$\begin{bmatrix} w_{11} & \cdots & w_{1N} \\ w_{21} & \cdots & w_{2N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}^{(l)} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(l-1)} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_M \end{bmatrix}^{(l)} = \begin{bmatrix} w_{11}x_1 + \cdots + w_{1N}x_N + b_1 \\ w_{21}x_1 + \cdots + w_{2N}x_N + b_2 \\ \vdots \\ w_{M1}x_1 + \cdots + w_{MN}x_N + b_M \end{bmatrix} \tag{1.5}$$

In other words, each neuron in layer l computes its own weighted sum over all outputs of the previous layer and then adds its corresponding bias term.

Equivalently, the bias can be absorbed into the matrix multiplication by augmenting the input vector with a constant entry 1 and extending the weight matrix by one additional column:

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_M^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & \cdots & w_{1N}^{(l)} & b_1^{(l)} \\ w_{21}^{(l)} & \cdots & w_{2N}^{(l)} & b_2^{(l)} \\ \vdots & \ddots & \vdots & \vdots \\ w_{M1}^{(l)} & \cdots & w_{MN}^{(l)} & b_M^{(l)} \end{bmatrix} \begin{bmatrix} x_1^{(l-1)} \\ x_2^{(l-1)} \\ \vdots \\ x_N^{(l-1)} \\ 1 \end{bmatrix} \tag{1.6}$$

Then the activation function is applied element-wise to the resulting pre-activation vector $\mathbf{z}^{(l)}$ to produce the output of layer l:

$$\mathbf{x}^{(l)} = \phi(\mathbf{z}^{(l)}) = \phi\left(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}\right). \tag{1.7}$$

Non-Linear Activation Functions in Neural Networks

The introduction of non-linear activation functions is essential, as a composition of purely linear transformations would again result in a linear model. By combining multiple layers with non-linear activations, neural networks can approximate highly complex functions and learn non-linear, intricate patterns from data. In fact, sufficiently large neural networks are universal function approximators, meaning they can approximate a wide range of continuous functions to arbitrary accuracy. Common activation functions used in modern neural networks include the functions GELU (Gaussian Error Linear Unit), ReLU (Rectified Linear Unit), Leaky ReLU, the tanh (hyperbolic tangent) and the Swish function. But there exist many more. Each of these functions has its own characteristics and can result in different learning dynamics and performance.

Prediction

The output layer is responsible for producing the network's final prediction. Its activation function is chosen based on the task:

- **Regression:** For regression problems, a linear activation directly outputs a continuous value.
- **Binary Classification:** For binary classification (true/false), a single neuron with a sigmoid function outputs a probability between 0 and 1.
- **Multi-Class Classification:** For multi-class classification, a softmax layer produces a probability distribution over the classes, where each output neuron corresponds to a specific class and the softmax function ensures that the outputs sum to 1, representing the predicted probabilities for each class.

The choice of activation function in the output layer is crucial, as it directly affects the interpretation of the network's predictions and the appropriate loss function to use during training.

Loss Function

The loss function $\mathcal{L}$ is a measure of how well the network is able to predict the target value and quantifies the difference between the network's predictions and the true labels in the training data. The goal of the training process is to minimize the loss function by adjusting the weights of the network. The loss function can be any differentiable function, but common choices are Mean Squared Error (MSE) for regression tasks and cross-entropy for classification tasks.

- **Mean Squared Error (MSE) Loss Function:** Commonly used for regression tasks, where the goal is to predict continuous values. The Mean Squared Error (MSE) loss function is defined as:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{1.8}$$

where y_i is the target value for the i-th sample, $\hat{y}_i$ is the predicted value for the i-th sample, and N is the number of samples.

- **Cross-Entropy Loss Function:** Commonly used for classification tasks, where the goal is to predict discrete class labels.
  - *Binary Classification:* For binary classification problems, the network typically uses a single output neuron with a sigmoid activation. The output $\hat{y}_i \in [0, 1]$ represents the predicted probability of the positive class. The binary cross-entropy loss is then defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)) \tag{1.9}$$

    where $y_i \in \{0, 1\}$ denotes the true label of the i-th sample, $\hat{y}_i$ is the predicted probability of the positive class, and N is the number of samples.
  - *Multi-Class Classification:* For multi-class classification problems with C classes, the network output is typically normalized using a softmax function to obtain a probability distribution over all classes (which sums up to 1 over the classes). The corresponding cross-entropy loss is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \sum_{i=1}^{C} y_{n,i} \log(\hat{y}_{n,i}) \tag{1.10}$$

    where $y_{n,i}$ represents the one-hot encoded ground-truth label and $\hat{y}_{n,i}$ is the predicted probability for class i of sample n.

1.1.3 Training Neural Networks

The parameters of a neural network, namely the weights and biases, are learned from data during training. This is typically achieved by minimizing a loss function $\mathcal{L}$ using optimization methods such as *gradient descent* and its variants. The gradients required for optimization are efficiently computed using the *backpropagation* algorithm, which propagates the prediction error from the output layer back through the network. This learning process enables neural networks to adapt their internal representations and achieve strong performance across a wide range of machine learning tasks.

What is a Gradient?

The gradient is a fundamental concept in optimization and machine learning. It describes how a function changes with respect to its input variables and points in the direction of the steepest increase of the function. In the special case of a function with only one variable, the gradient reduces to the first derivative, which corresponds to the slope of the function at a given point. In simple terms, the gradient indicates how the function value changes when making a small step to the right or left away from a specific point.

To better understand the concept of gradients, consider Figure 1.3. We want to determine the slope of the blue curve at the point $x = 1$. To do this, we calculate the gradient at that point. First, we compute the derivative $f'(x) = 2x$ of the function $f(x) = x^2 - 3$ and evaluate it at $x = 1$. The first derivative $f'(x)$ provides the slope at any point on the function $f(x)$. In this example, the slope at $x = 1$ is $f'(1) = 2$. To verify this result, the slope triangle is shown in Figure 1.3. It provides a visual representation of the slope and confirms the calculated value of with the slope $m = 2$.

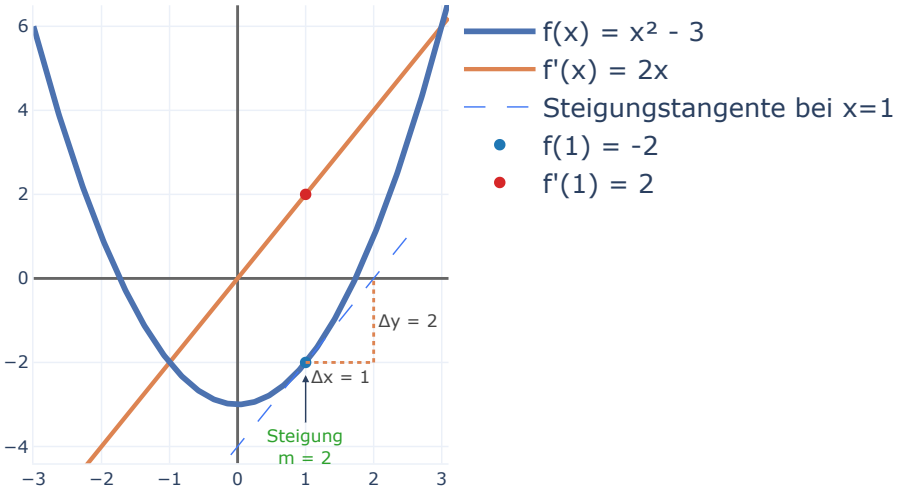

Figure 1.3: Neural Network Architecture.

In general, the gradient indicates whether a function is increasing or decreasing at a specific point and how steeply it changes. A positive gradient means that the function is increasing, while a negative gradient indicates that the function is decreasing. The magnitude of the gradient describes how steeply the function changes. Larger magnitudes correspond to steeper slopes, while values close to zero indicate relatively flat regions.

Gradient Descent and Parameter Update

Gradient descent is an iterative optimization algorithm that is used to minimize a loss function. The basic principle of gradient descent can be understood by considering a simplified setting with a single parameter—a network weight—as illustrated in the Figure 1.4. The horizontal axis represents the value of the weight, while the vertical axis represents the corresponding loss, which quantifies the discrepancy between the predicted and true values of the training data.

The resulting curve is the loss landscape for that one parameter. At any point on the loss curve, the derivative (or gradient) indicates the local slope of the function. The

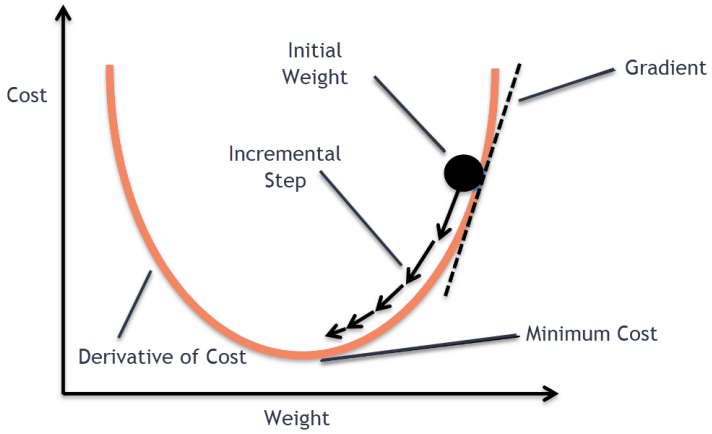

Figure 1.4: Neural Network Architecture. Image source: [**xxx**].

current value of the weight, represented by the black dot, must be adjusted such that the loss decreases. Gradient descent achieves this through the following two steps:

1. **Calculate the gradient:** Compute the gradient of the loss function with respect to the weight. The gradient indicates how the loss changes as the weight is adjusted.
2. **Update the weight:** The weight is then updated by moving in the opposite direction of the gradient. Since the gradient points towards the direction of steepest increase, moving against it results in a reduction of the loss. The basic parameter update is defined as:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \nabla \mathcal{L}(\mathbf{W}_t) \tag{1.11}$$

   where α denotes the learning rate, which controls the step size of the update, and $\nabla \mathcal{L}(\mathbf{W}_t)$ is the gradient of the loss function with respect to the parameter at iteration t.

In the example shown in Figure 1.4, the gradient is positive. Consequently, increasing the weight would increase the loss. To reduce the loss, the weight must therefore be moved in the opposite direction (to the left). This procedure is repeated iteratively until the algorithm reaches a minimum of the loss function or another stopping criterion is satisfied. The successive parameter updates are illustrated by the black arrows in Figure 1.4. At the minimum, the gradient and the slope are zero and the parameter update becomes zero as well. The algorithm has converged.

Goal of Training Neural Networks: The goal is to find the optimal set of parameters that minimizes the loss function, which corresponds to the best fit of the model to the data, meaning the network's predictions are sufficiently accurate.

Optimization Problems

But not every minimum found is a good solution. The loss landscape of neural networks is highly non-convex and can contain many local minima, saddle points, and flat regions, where the gradient is zero but the loss is not minimized. If the gradient

gets zero, the optimization process get stuck. Figure 1.4 shows a loss landscape with two parameters. The black dot represents the inital parameter values and the line shows the trajectory of the parameter updates during training. As seen it depends on the initial parameter values, whether the algorithm converges to the global minima, local minama or to the saddle point in the middle of the two hills.

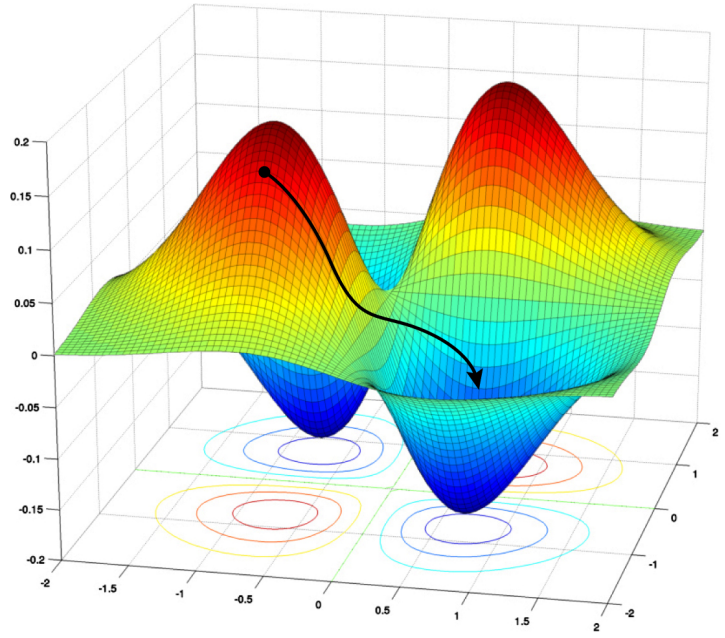

Figure 1.5: Neural Network Architecture. Image source: [**xxx**].

In the shown examples, the loss landscape was known, but in practice, the loss landscape of neural networks is typically unknown and can be highly complex. Neural networks often consists of millions of parameters and therefore they have a high-dimensional loss landscape. The Algorithm works in such high-dimensional spaces in the same way. Therefore, training neural networks can be challenging and may require careful tuning of hyperparameters such as the learning rate, as well as the use of techniques like momentum, adaptive learning rates or regularization to improve convergence and generalization performance. For this many variants of the basic gradient descent algorithm have been developed, which is not the scope of this work.

Backpropagation

The gradients required for updating the network parameters in gradient descent are computed using the *backpropagation algorithm*. Backpropagation efficiently propagates the prediction error from the output layer back through the network and computes the gradients of the loss function $\mathcal{L}$ with respect to all weights $\mathbf{W}$ in the model. Based on these gradients, each parameter w_{ij}^{l} is adjusted proportionally to its contribution to the overall error.

More precisely, backpropagation computes the partial derivatives of the loss function with respect to all trainable parameters in the network. These gradients quantify how

sensitive the loss is to small changes in each parameter and therefore indicate how each parameter should be adjusted to reduce the overall error. Parameters that contribute more strongly to the error receive larger updates, while less influential parameters are adjusted more mildly. The training process for a neural network can therefore be summarized by the following update rule:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \nabla_{\mathbf{W}_t} \left(\frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\mathbf{y}_i, f_{\mathbf{W}_t}(\mathbf{x}_i)\right) \right) \tag{1.12}$$

where $f_{\mathbf{W}}$ denotes the neural network with parameters $\mathbf{W}$, $\mathcal{L}$ is the loss function measuring the difference between the predicted output $f_{\mathbf{W}}(\mathbf{x}_i)$ and the target value $\mathbf{y}_i$, α is the learning rate, N is the number of training samples, and $\nabla_{\mathbf{W}_t}$ represents the gradient of the loss function with respect to the weights at iteration t.

By iteratively applying this update rule, the network parameters are gradually adjusted in a direction that reduces the loss function. This iterative optimization process is what allows the neural network to learn meaningful representations from training data and improve its predictions over time.

Besides the standard gradient descent algorithm, which computes the gradient over the entire dataset, several more efficient variants are commonly used in practice. In stochastic gradient descent (SGD), the exact gradient is not evaluated on the full dataset. Instead, it is approximated using only a single randomly selected training sample at each iteration. A further extension is mini-batch gradient descent, where the gradient is computed on a small subset of the training data (a mini-batch) rather than on a single sample or the full dataset. This approach reduces the computational cost per update while still providing a more stable gradient estimate compared to pure stochastic gradient descent.

1.1.4 Evaluation

Evaluation Metrics

To measure the performance of a machine learning model, various evaluation metrics can be used depending on the specific task and problem setting. These metrics provide quantitative measures of how well a model performs and allow comparisons between different models or configurations. While common examples include accuracy, precision, recall, and the F1-score for classification tasks, as well as mean squared error for regression tasks. However, a detailed discussion of these metrics is beyond the scope of this work.

Evaluation

The performance of a machine learning model is typically evaluated using data that has not been seen during training, referred to as test data. This allows an unbiased assessment of the model's ability to generalize to new, unseen samples. There exist in general three main scenarios regarding the model's performance on training and test data:

- **Underfitting:** A model that underfits performs poorly on both training and test data. It fails to capture the underlying patterns in the training data, resulting in high error on both datasets.
- **Overfitting:** An overfitted model achieves high performance on the training data but fails to generalize well to the test data. It has essentially memorized the training samples, including noise and outliers, which leads to poor performance on new data.
- **Well-Balanced Model:** A well-balanced model achieves consistently good performance on both training and test datasets. It captures the underlying patterns in the training data without overfitting, allowing it to generalize effectively to unseen samples.

—

—

Bibliography

[1] J.-W. Lin, "Artificial neural network related to biological neuron network: A review," 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:60402092.

[2] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biology*, vol. 52, pp. 99–115, 1990. [Online]. Available: https://api.semanticscholar.org/CorpusID:15619658.

[3] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain.," *Psychological review*, vol. 65 6, pp. 386–408, 1958. [Online]. Available: https://api.semanticscholar.org/CorpusID:12781225.

[4] M. Minsky *et al.*, "An introduction to computational geometry," 1969. [Online]. Available: https://api.semanticscholar.org/CorpusID:118679952.

List of Figures

1.1 Artificial Neuron 2
1.2 Neural Network Architecture 4
1.3 Neural Network Architecture 7
1.4 Neural Network Architecture 8
1.5 Neural Network Architecture 9

List of Tables